DAS7001 Practical Assessment: Database Systems

Student ID: st20310831

Contents

ERD (Entity Relationship Diagram)

Data Dictionaries

- 2.1 Department Table
- 2.2 Staff Table
- 2.3 Applicant Table
- 2.4 ApplicantAddress Table
- 2.5 ApplicantPhone Table
- 2.6 Course Table
- 2.7 Application Table (Updated)
- 2.8 Document Table
- 2.9 ApplicationReview Table

Implementation using Microsoft SQL Server Express (Task 1.3)

SQL Queries (Task 1.4)

- 4.1 INSERT Query
- 4.2 Aggregate Function Query
- 4.3 Wildcard Operator Query
- 4.4 INNER JOIN Query
- 4.5 FULL OUTER JOIN Query

1. ERD (Entity Relationship Diagram)

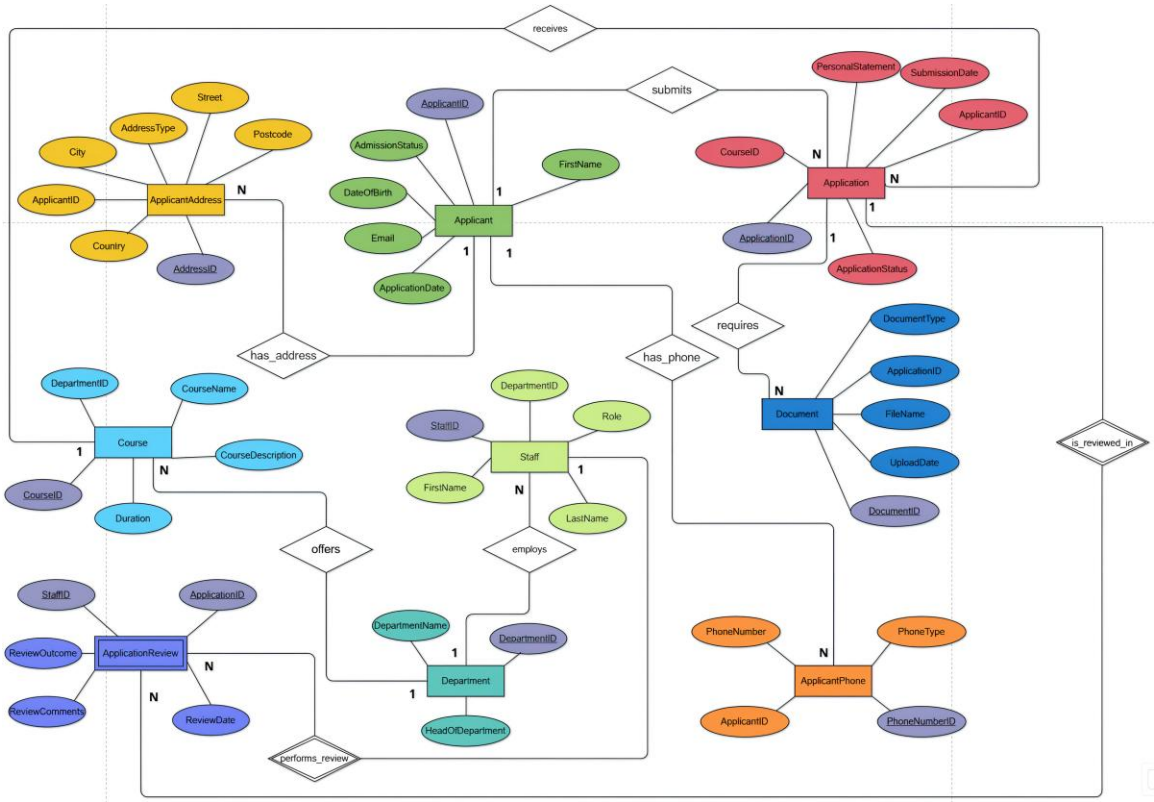

Figure 1: Chen Notation ERD for the Student Admission Database System.

2. Data Dictionaries

The following data dictionaries detail the structure of each table in the Student Admission Database System.

2.1 Department Table

Description: Stores information about academic departments within the institution.

Attribute Name	Data Type	Length	Constraints	Description
DepartmentID	INT		PK, NOT NULL	Unique identifier for each department
DepartmentName	VARCHAR	100	NOT NULL, UNIQUE	Name of the academic

				department
HeadOfDepartment	INT		FK references Staff (StaffID), NULL	Identifier for the staff member leading the department

2.2 Staff Table

Description: Stores information about staff members, including admission officers.

Attribute Name	Data Type	Length	Constraints	Description
StaffID	INT		PK, NOT NULL	Unique identifier for each staff member
FirstName	VARCHAR	50	NOT NULL	Staff member's first name
LastName	VARCHAR	50	NOT NULL	Staff member's last name
DepartmentID	INT		FK references Department (DepartmentID), NOT NULL	Department the staff member belongs to
Role	VARCHAR	50	NULL	Role of the staff member (e.g., 'Admissions Officer', 'HOD')

2.3 Applicant Table

Description: Stores primary information about individuals applying to the institution.

Attribute Name	Data Type	Length	Constraints	Description
ApplicantID	INT		PK, NOT NULL	Unique identifier for each applicant
FirstName	VARCHAR	50	NOT NULL	Applicant's first name
LastName	VARCHAR	50	NOT NULL	Applicant's last name
DateOfBirth	DATE		NULL	Applicant's date of birth
Email	VARCHAR	100	UNIQUE, NULL	Applicant's

				primary email address
ApplicationDate	DATE		NULL, DEFAULT GETDATE()	Date the applicant record was created
AdmissionStatus	VARCHAR	10	CHECK (AdmissionStatus IN ('Pending', 'Accepted', 'Rejected')), NULL	Current overall admission status for the applicant

2.4 ApplicantAddress Table

Description: Stores address information for applicants. Allows for multiple addresses per applicant.

Attribute Name	Data Type	Length	Constraints	Description
AddressID	INT		PK, NOT NULL, IDENTITY(1,1)	Unique identifier for each address record
ApplicantID	INT		FK references Applicant(ApplicantID), NOT NULL, ON DELETE CASCADE	Links the address to an applicant
AddressType	VARCHAR	10	CHECK (AddressType IN ('Home', 'Mailing')), NULL	Type of address (e.g., Home, Mailing)
Street	VARCHAR	100	NULL	Street address line
City	VARCHAR	50	NULL	City name
Postcode	VARCHAR	10	NULL	Postal code
Country	VARCHAR	50	NULL, DEFAULT 'UK'	Country name

2.5 ApplicantPhone Table

Description: Stores phone number information for applicants. Allows for multiple numbers per applicant.

Attribute Name	Data Type	Length	Constraints	Description
PhoneNumberID	INT		PK, NOT NULL, IDENTITY(1,1)	Unique identifier for

				each phone number record
ApplicantID	INT		FK references Applicant(ApplicantID), NOT NULL, ON DELETE CASCADE	Links the phone number to an applicant
PhoneType	VARCHAR	10	CHECK (PhoneType IN ('Home', 'Mobile')), NULL	Type of phone number (e.g., Home, Mobile)
PhoneNumber	VARCHAR	20	NULL	The phone number

2.6 Course Table

Description: Stores information about courses offered by the institution.

Attribute Name	Data Type	Length	Constraints	Description
CourseID	INT		PK, NOT NULL	Unique identifier for each course
CourseName	VARCHAR	100	NOT NULL, UNIQUE	Name of the course
DepartmentID	INT		FK references Department(DepartmentID), NOT NULL	Department offering the course
CourseDescription	VARCHAR	255	NULL	Brief description of the course
Duration	VARCHAR	20	NULL	Duration of the course (e.g., '3 Years', '1 Year')

2.7 Application Table

Description: Stores details about each application submitted by an applicant for a specific course.

Attribute Name	Data Type	Length	Constraints	Description
ApplicationID	INT		PK, NOT NULL	Unique identifier for each application

ApplicantID	INT		FK references Applicant(ApplicantID), NOT NULL, ON DELETE CASCADE	Applicant who submitted the application
CourseID	INT		FK references Course(CourseID), NOT NULL	Course the application is for
SubmissionDate	DATETIME		NOT NULL, DEFAULT GETDATE()	Date and time the application was submitted
PersonalStatement	VARCHAR(MAX)		NULL	Text of the applicant's personal statement
ApplicationStatus	VARCHAR	25	CHECK (ApplicationStatus IN ('Submitted', 'Under Review', 'Awaiting Documents', 'Complete', 'Decision Made', 'Offer Accepted', 'Withdrawn')), NOT NULL, DEFAULT 'Submitted'	Current status of this specific application

2.8 Document Table

Description: Stores information about supporting documents submitted with an application.

Attribute Name	Data Type	Length	Constraints	Description
DocumentID	INT		PK, NOT NULL, IDENTITY(1,1)	Unique identifier for each document
ApplicationID	INT		FK references Application(ApplicationID), NOT NULL, ON DELETE CASCADE	Application the document belongs to
DocumentType	VARCHAR	50	NOT NULL	Type of document (e.g.,

				'Transcript', 'Reference')
FileName	VARCHAR	100	NOT NULL	Original filename of the uploaded document
UploadDate	DATETIME		NOT NULL, DEFAULT GETDATE()	Date and time the document was uploaded

2.9 ApplicationReview Table (Weak Entity)

Description: Linking table for the M:N relationship between Staff and Application, recording review details.

Attribute Name	Data Type	Length	Constraints	Description
StaffID	INT		PK, FK references Staff(StaffID), NOT NULL	Staff member performing the review
ApplicationID	INT		PK, FK references Application(ApplicationID), NOT NULL	Application being reviewed
ReviewDate	DATETIME		NOT NULL, DEFAULT GETDATE()	Date and time the review was conducted/recorded
ReviewComments	VARCHAR(MAX)		NULL	Comments made by the reviewer
ReviewOutcome	VARCHAR	25	CHECK (ReviewOutcome IN ('Positive', 'Negative', 'Neutral', 'Interview Recommended')), NULL	Outcome or recommendation from this review step

3. Implementation using Microsoft SQL Server Express (Task 1.3)

The following SQL script creates the StudentAdmissionDB database, defines the table structures based on the ERD and Data Dictionaries, establishes constraints (Primary Keys, Foreign Keys, CHECK constraints, Defaults), and inserts sample data. Examples of basic CRUD operations are included at the end.

```sql
-- Create the database
PRINT '==========================================';
PRINT 'Section 1: Creating Database StudentAdmissionDB...';
PRINT '==========================================';
CREATE DATABASE StudentAdmissionDB;
GO

-- Switch context to the new database
USE StudentAdmissionDB;
GO
PRINT 'Database created and context switched to StudentAdmissionDB.';
GO

-- ==========================================
-- Section 2: Table Creation
-- Tables created in order to satisfy FK constraints.
-- ==========================================
PRINT '==========================================';
PRINT 'Section 2: Creating Tables...';
PRINT '==========================================';

-- Table: Department
PRINT 'Creating Table: Department...';
CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY NOT NULL,              -- Unique identifier for each department
    DepartmentName VARCHAR(100) NOT NULL UNIQUE        -- Name of the academic department
    -- HeadOfDepartment FK constraint added later after Staff table
);
GO

-- Table: Staff
PRINT 'Creating Table: Staff...';
CREATE TABLE Staff (
    StaffID INT PRIMARY KEY NOT NULL,                  -- Unique identifier for each staff member
    FirstName VARCHAR(50) NOT NULL,                    -- Staff member's first name
    LastName VARCHAR(50) NOT NULL,                     -- Staff member's last name
    DepartmentID INT NOT NULL,                         -- Department the staff member belongs to
    Role VARCHAR(50) NULL,                             -- Role of the staff member
    CONSTRAINT FK_Staff_Department FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
GO

-- Add FK constraint for HeadOfDepartment to Department table
PRINT 'Adding FK constraint FK_Department_Head to Department...';
ALTER TABLE Department
ADD HeadOfDepartment INT NULL,                         -- FK references Staff(StaffID)
    CONSTRAINT FK_Department_Head FOREIGN KEY (HeadOfDepartment) REFERENCES Staff(StaffID);
GO

-- Table: Applicant
PRINT 'Creating Table: Applicant...';
CREATE TABLE Applicant (
    ApplicantID INT PRIMARY KEY NOT NULL,              -- Unique identifier for each applicant
    FirstName VARCHAR(50) NOT NULL,                    -- Applicant's first name
    LastName VARCHAR(50) NOT NULL,                     -- Applicant's last name
    DateOfBirth DATE NULL,                             -- Applicant's date of birth
    Email VARCHAR(100) UNIQUE NULL,                    -- Applicant's primary email address
    ApplicationDate DATE NULL DEFAULT GETDATE(),       -- Date the applicant record was created
    AdmissionStatus VARCHAR(10) NULL                   -- Overall admission status for the applicant
        CHECK (AdmissionStatus IN ('Pending', 'Accepted', 'Rejected'))
);
GO
```

```sql
-- Table: ApplicantAddress
PRINT 'Creating Table: ApplicantAddress...';
CREATE TABLE ApplicantAddress (
    AddressID INT PRIMARY KEY NOT NULL IDENTITY(1,1),   -- Auto-incrementing PK
    ApplicantID INT NOT NULL,                           -- FK references Applicant(ApplicantID)
    AddressType VARCHAR(10) NULL                        -- Type of address (e.g., Home, Mailing)
        CHECK (AddressType IN ('Home', 'Mailing')),
    Street VARCHAR(100) NULL,                           -- Street address line
    City VARCHAR(50) NULL,                              -- City name
    Postcode VARCHAR(10) NULL,                          -- Postal code
    Country VARCHAR(50) NULL DEFAULT 'UK',              -- Country name (Default added)
    CONSTRAINT FK_ApplicantAddress_Applicant FOREIGN KEY (ApplicantID) REFERENCES Applicant(ApplicantID) ON DELETE CASCADE
);
GO

-- Table: ApplicantPhone
PRINT 'Creating Table: ApplicantPhone...';
CREATE TABLE ApplicantPhone (
    PhoneNumberID INT PRIMARY KEY NOT NULL IDENTITY(1,1), -- Auto-incrementing PK
    ApplicantID INT NOT NULL,                           -- FK references Applicant(ApplicantID)
    PhoneType VARCHAR(10) NULL                          -- Type of phone (e.g., Home, Mobile)
        CHECK (PhoneType IN ('Home', 'Mobile')),
    PhoneNumber VARCHAR(20) NULL,                       -- The phone number
    CONSTRAINT FK_ApplicantPhone_Applicant FOREIGN KEY (ApplicantID) REFERENCES Applicant(ApplicantID) ON DELETE CASCADE
);
GO

-- Table: Course
PRINT 'Creating Table: Course...';
CREATE TABLE Course (
    CourseID INT PRIMARY KEY NOT NULL,                  -- Unique identifier for each course
    CourseName VARCHAR(100) NOT NULL UNIQUE,            -- Name of the course
    DepartmentID INT NOT NULL,                          -- FK references Department(DepartmentID)
    CourseDescription VARCHAR(255) NULL,                -- Brief description of the course
    Duration VARCHAR(20) NULL,                          -- Duration of the course
    CONSTRAINT FK_Course_Department FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
GO

-- Table: Application
PRINT 'Creating Table: Application...';
CREATE TABLE Application (
    ApplicationID INT PRIMARY KEY NOT NULL,             -- Unique identifier for each application
    ApplicantID INT NOT NULL,                           -- FK references Applicant(ApplicantID)
    CourseID INT NOT NULL,                              -- FK references Course(CourseID)
    SubmissionDate DATETIME NOT NULL DEFAULT GETDATE(), -- Date and time submitted
    PersonalStatement VARCHAR(MAX) NULL,                -- Applicant's personal statement text
    ApplicationStatus VARCHAR(25) NOT NULL DEFAULT 'Submitted' -- Status of this application -- <<< *** CORRECTED LENGTH ***
        CHECK (ApplicationStatus IN ('Submitted', 'Under Review', 'Awaiting Documents', 'Complete', 'Decision Made', 'Offer Accepted', 'Withdrawn')),
    CONSTRAINT FK_Application_Applicant FOREIGN KEY (ApplicantID) REFERENCES Applicant(ApplicantID) ON DELETE CASCADE,
    CONSTRAINT FK_Application_Course FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
GO

-- Table: Document
PRINT 'Creating Table: Document...';
CREATE TABLE Document (
    DocumentID INT PRIMARY KEY NOT NULL IDENTITY(1,1),  -- Auto-incrementing PK
    ApplicationID INT NOT NULL,                         -- FK references Application(ApplicationID)
    DocumentType VARCHAR(50) NOT NULL,                  -- Type of document (e.g., 'Transcript', 'Reference')
    FileName VARCHAR(100) NOT NULL,                     -- Original filename
    UploadDate DATETIME NOT NULL DEFAULT GETDATE(),     -- Date and time uploaded
    CONSTRAINT FK_Document_Application FOREIGN KEY (ApplicationID) REFERENCES Application(ApplicationID) ON DELETE CASCADE
);
GO
```

```sql
-- Table: ApplicationReview
PRINT 'Creating Table: ApplicationReview...';
CREATE TABLE ApplicationReview (
    StaffID INT NOT NULL,                           -- FK references Staff(StaffID)
    ApplicationID INT NOT NULL,                     -- FK references Application(ApplicationID)
    ReviewDate DATETIME NOT NULL DEFAULT GETDATE(),     -- Date and time review recorded
    ReviewComments VARCHAR(MAX) NULL,               -- Reviewer's comments
    ReviewOutcome VARCHAR(25) NULL                  -- Outcome/recommendation -- Increased length slightly for safety
        CHECK (ReviewOutcome IN ('Positive', 'Negative', 'Neutral', 'Interview Recommended')),
    CONSTRAINT PK_ApplicationReview PRIMARY KEY (StaffID, ApplicationID), -- Composite Primary Key
    CONSTRAINT FK_ApplicationReview_Staff FOREIGN KEY (StaffID) REFERENCES Staff(StaffID),
    CONSTRAINT FK_ApplicationReview_Application FOREIGN KEY (ApplicationID) REFERENCES Application(ApplicationID) --
);
GO

PRINT 'All tables created successfully.';
GO

-- ============================================
-- Section 3: Data Insertion
-- Inserting at least 10 records where applicable.
-- ============================================
PRINT '============================================';
PRINT 'Section 3: Inserting Sample Data...';
PRINT '============================================';
GO

-- Departments (Adding a few more)
PRINT 'Inserting Departments...';
INSERT INTO Department (DepartmentID, DepartmentName) VALUES
(1, 'School of Computing & Engineering'),
(2, 'School of Business Management'),
(3, 'School of Art & Design'),
(4, 'School of Health Sciences'),
(5, 'School of Social Sciences');
GO

-- Staff (Expanding to 10+ records)
PRINT 'Inserting Staff...';
INSERT INTO Staff (StaffID, FirstName, LastName, DepartmentID, Role) VALUES
(101, 'Eleanor', 'Rigby', 1, 'Admissions Tutor'),
(102, 'Desmond', 'Jones', 2, 'Senior Lecturer'),
(103, 'Lucy', 'Diamond', 1, 'Head of School'),
(104, 'Maxwell', 'Edison', 3, 'Admissions Officer'),
(105, 'Rita', 'Metermaid', 2, 'Administrator'),
(106, 'Michelle', 'Belle', 3, 'Lecturer'),
(107, 'Julia', 'Lennon', 1, 'Researcher'),
(108, 'Prudence', 'Dear', 4, 'Admissions Head'),
(109, 'Martha', 'MyDear', 5, 'Senior Tutor'),
(110, 'Polythene', 'Pam', 1, 'Technician'),
(111, 'Sun', 'King', 4, 'Lecturer');
GO

-- Update HOD in Department
PRINT 'Updating Department HeadOfDepartment FKs...';
UPDATE Department SET HeadOfDepartment = 103 WHERE DepartmentID = 1;
UPDATE Department SET HeadOfDepartment = 108 WHERE DepartmentID = 4;
-- Add others if needed
GO
```

```sql
-- Applicants (Expanding to 10+ records)
PRINT 'Inserting Applicants...';
INSERT INTO Applicant (ApplicantID, FirstName, LastName, DateOfBirth, Email, AdmissionStatus) VALUES
(2001, 'Jude', 'Hey', '2005-04-01', 'jude.h@email.com', 'Pending'),
(2002, 'Penny', 'Lane', '2004-11-15', 'p.lane@email.com', 'Accepted'),
(2003, 'Rocky', 'Raccoon', '2006-01-20', 'rocky.r@email.com', 'Pending'),
(2004, 'Eleanor', 'Vance', '2005-07-22', 'e.vance@email.com', 'Pending'),
(2005, 'Maxwell', 'Silver', '2004-09-03', 'm.silver@email.com', 'Rejected'),
(2006, 'Lucy', 'Sky', '2006-03-10', 'lucy.s@email.com', 'Pending'),
(2007, 'Desmond', 'Morris', '2005-05-18', 'dez.morris@email.com', 'Accepted'),
(2008, 'Rita', 'Wright', '2004-12-25', 'rita.w@email.com', 'Pending'),
(2009, 'Julia', 'Baird', '2006-02-14', 'julia.b@email.com', 'Pending'),
(2010, 'Martha', 'Jones', '2005-08-30', 'martha.j@email.com', 'Pending'),
(2011, 'Prudence', 'Farrow', '2004-06-06', 'prudence.f@email.com', 'Pending');
GO

-- Applicant Addresses (Adding more, ensuring FKs match Applicants)
PRINT 'Inserting Applicant Addresses...';
INSERT INTO ApplicantAddress (ApplicantID, AddressType, Street, City, Postcode, Country) VALUES
(2001, 'Home', '1 Abbey Road', 'London', 'NW8 9AY', 'UK'),
(2002, 'Mailing', '2 Penny Lane', 'Liverpool', 'L18 1DE', 'UK'),
(2002, 'Home', '3 Beatles Drive', 'Liverpool', 'L1 0AX', 'UK'),
(2003, 'Home', '4 Rocky Road', 'Dakota', 'DA1 5TA', 'USA'),
(2004, 'Home', '5 Eleanor Close', 'Cardiff', 'CF10 3AA', 'UK'),
(2005, 'Home', '6 Maxwell St', 'Manchester', 'M1 1AA', 'UK'),
(2006, 'Home', '7 Lucy Lane', 'Bristol', 'BS1 1AB', 'UK'),
(2007, 'Mailing', '8 Desmond Drive', 'Birmingham', 'B1 1AC', 'UK'),
(2008, 'Home', '9 Rita Road', 'London', 'E1 6AN', 'UK'),
(2009, 'Home', '10 Julia Close', 'Liverpool', 'L15 6AS', 'UK'),
(2010, 'Home', '11 Martha Mews', 'Cardiff', 'CF24 4AX', 'UK'),
(2011, 'Mailing', '12 Prudence Place', 'Glasgow', 'G1 1AD', 'UK');
GO

-- Applicant Phone Numbers (Adding more)
PRINT 'Inserting Applicant Phone Numbers...';
INSERT INTO ApplicantPhone (ApplicantID, PhoneType, PhoneNumber) VALUES
(2001, 'Mobile', '07700900001'),
(2002, 'Mobile', '07700900002'),
(2003, 'Home', '0151 496 0003'),
(2004, 'Mobile', '07700900004'),
(2005, 'Mobile', '07700900005'),
(2006, 'Home', '0117 496 0006'),
(2007, 'Mobile', '07700900007'),
(2008, 'Mobile', '07700900008'),
(2009, 'Home', '0151 496 0009'),
(2010, 'Mobile', '07700900010'),
(2011, 'Mobile', '07700900011');
GO

-- Courses (Adding more)
PRINT 'Inserting Courses...';
INSERT INTO Course (CourseID, CourseName, DepartmentID, Duration) VALUES
(101, 'BSc Computer Science', 1, '3 Years'),
(102, 'MSc Data Science', 1, '1 Year'),
(103, 'BSc Cyber Security', 1, '3 Years'),
(201, 'BA Business Studies', 2, '3 Years'),
(202, 'MBA', 2, '1 Year'),
(301, 'BA Fine Art', 3, '3 Years'),
(302, 'BA Graphic Design', 3, '3 Years'),
(401, 'BSc Nursing', 4, '3 Years'),
(402, 'MSc Public Health', 4, '1 Year'),
(501, 'BA Sociology', 5, '3 Years');
GO
```

```sql
-- Applicants (Expanding to 10+ records)
PRINT 'Inserting Applicants...';
INSERT INTO Applicant (ApplicantID, FirstName, LastName, DateOfBirth, Email, AdmissionStatus) VALUES
(2001, 'Jude', 'Hey', '2005-04-01', 'jude.h@email.com', 'Pending'),
(2002, 'Penny', 'Lane', '2004-11-15', 'p.lane@email.com', 'Accepted'),
(2003, 'Rocky', 'Raccoon', '2006-01-20', 'rocky.r@email.com', 'Pending'),
(2004, 'Eleanor', 'Vance', '2005-07-22', 'e.vance@email.com', 'Pending'),
(2005, 'Maxwell', 'Silver', '2004-09-03', 'm.silver@email.com', 'Rejected'),
(2006, 'Lucy', 'Sky', '2006-03-10', 'lucy.s@email.com', 'Pending'),
(2007, 'Desmond', 'Morris', '2005-05-18', 'dez.morris@email.com', 'Accepted'),
(2008, 'Rita', 'Wright', '2004-12-25', 'rita.w@email.com', 'Pending'),
(2009, 'Julia', 'Baird', '2006-02-14', 'julia.b@email.com', 'Pending'),
(2010, 'Martha', 'Jones', '2005-08-30', 'martha.j@email.com', 'Pending'),
(2011, 'Prudence', 'Farrow', '2004-06-06', 'prudence.f@email.com', 'Pending');
GO

-- Applicant Addresses (Adding more, ensuring FKs match Applicants)
PRINT 'Inserting Applicant Addresses...';
INSERT INTO ApplicantAddress (ApplicantID, AddressType, Street, City, Postcode, Country) VALUES
(2001, 'Home', '1 Abbey Road', 'London', 'NW8 9AY', 'UK'),
(2002, 'Mailing', '2 Penny Lane', 'Liverpool', 'L18 1DE', 'UK'),
(2002, 'Home', '3 Beatles Drive', 'Liverpool', 'L1 0AX', 'UK'),
(2003, 'Home', '4 Rocky Road', 'Dakota', 'DA1 5TA', 'USA'),
(2004, 'Home', '5 Eleanor Close', 'Cardiff', 'CF10 3AA', 'UK'),
(2005, 'Home', '6 Maxwell St', 'Manchester', 'M1 1AA', 'UK'),
(2006, 'Home', '7 Lucy Lane', 'Bristol', 'BS1 1AB', 'UK'),
(2007, 'Mailing', '8 Desmond Drive', 'Birmingham', 'B1 1AC', 'UK'),
(2008, 'Home', '9 Rita Road', 'London', 'E1 6AN', 'UK'),
(2009, 'Home', '10 Julia Close', 'Liverpool', 'L15 6AS', 'UK'),
(2010, 'Home', '11 Martha Mews', 'Cardiff', 'CF24 4AX', 'UK'),
(2011, 'Mailing', '12 Prudence Place', 'Glasgow', 'G1 1AD', 'UK');
GO

-- Applicant Phone Numbers (Adding more)
PRINT 'Inserting Applicant Phone Numbers...';
INSERT INTO ApplicantPhone (ApplicantID, PhoneType, PhoneNumber) VALUES
(2001, 'Mobile', '07700900001'),
(2002, 'Mobile', '07700900002'),
(2003, 'Home', '0151 496 0003'),
(2004, 'Mobile', '07700900004'),
(2005, 'Mobile', '07700900005'),
(2006, 'Home', '0117 496 0006'),
(2007, 'Mobile', '07700900007'),
(2008, 'Mobile', '07700900008'),
(2009, 'Home', '0151 496 0009'),
(2010, 'Mobile', '07700900010'),
(2011, 'Mobile', '07700900011');
GO

-- Courses (Adding more)
PRINT 'Inserting Courses...';
INSERT INTO Course (CourseID, CourseName, DepartmentID, Duration) VALUES
(101, 'BSc Computer Science', 1, '3 Years'),
(102, 'MSc Data Science', 1, '1 Year'),
(103, 'BSc Cyber Security', 1, '3 Years'),
(201, 'BA Business Studies', 2, '3 Years'),
(202, 'MBA', 2, '1 Year'),
(301, 'BA Fine Art', 3, '3 Years'),
(302, 'BA Graphic Design', 3, '3 Years'),
(401, 'BSc Nursing', 4, '3 Years'),
(402, 'MSc Public Health', 4, '1 Year'),
(501, 'BA Sociology', 5, '3 Years');
GO
```

```sql
-- Applications (Adding more, ensure ApplicantID/CourseID exist)
PRINT 'Inserting Applications...';
INSERT INTO Application (ApplicationID, ApplicantID, CourseID, ApplicationStatus, SubmissionDate) VALUES
(5001, 2001, 101, 'Submitted', '2025-01-15 10:30:00'),
(5002, 2002, 201, 'Offer Accepted', '2025-01-20 14:00:00'),
(5003, 2003, 101, 'Submitted', '2025-02-01 09:15:00'),
(5004, 2001, 102, 'Submitted', '2025-02-05 11:00:00'),
(5005, 2004, 401, 'Under Review', '2025-02-10 16:00:00'),
(5006, 2005, 301, 'Decision Made', '2025-02-12 10:00:00'),
(5007, 2006, 302, 'Submitted', '2025-02-18 13:20:00'),
(5008, 2007, 202, 'Offer Accepted', '2025-02-20 09:00:00'),
(5009, 2008, 501, 'Awaiting Documents', '2025-02-25 15:00:00'),
(5010, 2009, 103, 'Submitted', '2025-03-01 11:45:00'),
(5011, 2010, 401, 'Under Review', '2025-03-05 10:10:00'),
(5012, 2011, 101, 'Submitted', '2025-03-10 12:00:00');
GO

-- Documents (Adding more, ensure ApplicationID exists)
PRINT 'Inserting Documents...';
INSERT INTO Document (ApplicationID, DocumentType, FileName) VALUES
(5001, 'Transcript', 'jude_transcript.pdf'),
(5001, 'Reference', 'jude_reference1.pdf'),
(5002, 'Transcript', 'penny_results.pdf'),
(5003, 'Personal Statement Copy', 'rocky_ps_final.docx'),
(5004, 'CV', 'jude_cv_msc.pdf'),
(5005, 'Reference', 'eleanor_ref.pdf'),
(5005, 'Transcript', 'eleanor_transcript.pdf'),
(5006, 'Portfolio', 'maxwell_portfolio.zip'),
(5007, 'GCSE Certificates', 'lucy_certs.pdf'),
(5008, 'Degree Certificate', 'desmond_mba_cert.pdf'),
(5009, 'Reference', 'rita_reference.pdf'), -- App 5009 awaiting docs, maybe transcript missing
(5010, 'Transcript', 'julia_transcript.pdf'),
(5011, 'Reference', 'martha_ref.docx'),
(5012, 'IELTS Certificate', 'prudence_ielts.pdf');
GO

-- Application Reviews (Adding more, ensure StaffID/ApplicationID exist)
PRINT 'Inserting Application Reviews...';
INSERT INTO ApplicationReview (StaffID, ApplicationID, ReviewComments, ReviewOutcome) VALUES
(101, 5001, 'Good academic record, suitable candidate.', 'Positive'),
(102, 5002, 'Relevant work experience, check references.', 'Positive'),
(104, 5003, 'Portfolio review needed.', 'Neutral'),
(101, 5004, 'Strong candidate for MSc.', 'Positive'),
(108, 5005, 'Meets entry requirements, interview maybe?', 'Interview Recommended'),
(104, 5006, 'Portfolio does not meet standard.', 'Negative'),
(111, 5007, 'Standard application.', 'Neutral'),
(102, 5008, 'Excellent candidate for MBA.', 'Positive'),
(109, 5009, 'Awaiting transcript before review.', 'Neutral'),
(101, 5010, 'Solid grades, check cyber interest.', 'Positive'),
(108, 5011, 'Good personal statement, check qualifications.', 'Neutral'),
(103, 5012, 'Excellent grades, relevant project work mentioned.', 'Positive'); -- HOD review
GO


PRINT 'Sample data insertion complete.';
GO
```

4. SQL Queries (Task 1.4)

This section provides the five specific SQL queries required by the assessment brief, demonstrating various SQL operations on the StudentAdmissionDB database.

4.1 INSERT Query

Purpose: Demonstrates adding new records...

```sql
-- ==========================================
-- Query 1: INSERT Example
-- Requirement: Write a SQL query which inserts a set of values into a table.
-- Action: Insert a new review record for an existing application and staff member.
--         Assumes ApplicationID 5012 and StaffID 101 exist.
-- ==========================================
INSERT INTO ApplicationReview (StaffID, ApplicationID, ReviewComments, ReviewOutcome)
VALUES (101, 5012, 'Further review positive, meets all criteria.', 'Positive');

-- Verify the insertion (optional - you can run this separately if needed)
SELECT * FROM ApplicationReview WHERE ApplicationID = 5012 AND StaffID = 101;

GO
```

%

Results | Messages

StaffID	ApplicationID	ReviewDate	ReviewComments	ReviewOutcome
101	5012	2025-04-22 12:41:39.057	Further review positive, meets all criteria.	Positive

4.2 Aggregate Function Query

Purpose: Demonstrates summarizing data...

```sql
-- ==========================================
-- Query 2: Aggregate Function with GROUP BY and Alias Example
-- Requirement: Write a SQL query showing the aggregate function making use of group by and alias.
-- Action: Count the number of applications received for each course.
-- ==========================================
SELECT
    C.CourseName,                  -- Select the course name from the Course table
    COUNT(A.ApplicationID) AS NumberOfApplications -- Count the applications for each course and name the result 'NumberOfApplications'
FROM
    Application AS A               -- Alias the Application table as 'A'
JOIN
    Course AS C ON A.CourseID = C.CourseID -- Join Application and Course tables on CourseID
GROUP BY
    C.CourseName                   -- Group the results by course name to count applications per course
ORDER BY
    NumberOfApplications DESC;     -- Order the results to show courses with most applications first

GO
```

100 %

Results | Messages

	CourseName	NumberOfApplications
1	BSc Computer Science	3
2	BSc Nursing	2
3	MBA	1
4	MSc Data Science	1
5	BSc Cyber Security	1
6	BA Business Studies	1
7	BA Fine Art	1
8	BA Graphic Design	1
9	BA Sociology	1

4.3 Wildcard Operator Query

Purpose: Demonstrates selecting data based on pattern matching...

```sql
-- ==============================================
-- Query 3: Wildcard Operator Example
-- Requirement: Write a SQL query which shows few record(s) using Wildcard operator(s).
-- Action: Find all staff members whose Role contains the word 'Officer'.
-- ==============================================
SELECT
    StaffID,
    FirstName,
    LastName,
    Role
FROM
    Staff
WHERE
    Role LIKE '%Officer%'; -- Select rows where the Role column contains 'Officer' anywhere within it

GO

-- ==============================================
-- Query 4: INNER JOIN Example
-- Requirement: Write a SQL query showing Inner Join between 2 tables.
```

100 %

Results | Messages

	StaffID	FirstName	LastName	Role
1	104	Maxwell	Edison	Admissions Officer

4.4 INNER JOIN Query

Purpose: Demonstrates combining related rows...

```sql
-- ==============================================
-- Query 4: INNER JOIN Example
-- Requirement: Write a SQL query showing Inner Join between 2 tables.
-- Action: List the names of applicants and the submission date of their application(s).
-- ==============================================
SELECT
    App.FirstName,          -- Applicant's first name
    App.LastName,           -- Applicant's last name
    A.ApplicationID,        -- The Application ID
    A.SubmissionDate,       -- The date the application was submitted
    A.ApplicationStatus     -- The status of that specific application
FROM
    Applicant AS App        -- Alias the Applicant table as 'App'
INNER JOIN
    Application AS A ON App.ApplicantID = A.ApplicantID; -- Join based on matching ApplicantID in both tables

GO

-- ==============================================
```

100 %

Results | Messages

	FirstName	LastName	ApplicationID	SubmissionDate	ApplicationStatus
1	Jude	Hey	5001	2025-01-15 10:30:00.000	Submitted
2	Penny	Lane	5002	2025-01-20 14:00:00.000	Offer Accepted
3	Rocky	Raccoon	5003	2025-02-01 09:15:00.000	Complete
4	Jude	Hey	5004	2025-02-05 11:00:00.000	Submitted
5	Eleanor	Vance	5005	2025-02-10 16:00:00.000	Under Review
6	Maxwell	Silver	5006	2025-02-12 10:00:00.000	Decision Made
7	Lucy	Sky	5007	2025-02-18 13:20:00.000	Submitted
8	Desmond	Morris	5008	2025-02-20 09:00:00.000	Offer Accepted
9	Rita	Wright	5009	2025-02-25 15:00:00.000	Awaiting Documents
10	Julia	Baird	5010	2025-03-01 11:45:00.000	Submitted
11	Martha	Jones	5011	2025-03-05 10:10:00.000	Under Review
12	Prudence	Farrow	5012	2025-03-10 12:00:00.000	Submitted

4.5 FULL OUTER JOIN Query

Purpose: Demonstrates combining rows from two tables, including all rows...

```sql
-- ==========================================
-- Query 5: FULL OUTER JOIN Example
-- Requirement: Write a SQL query showing full Outer Join between 2 tables.
-- Action: Show all Applicants and all their Phone Numbers, matching where possible.
-- ==========================================
SELECT
    A.ApplicantID,
    A.FirstName,
    A.LastName,
    AP.PhoneType,
    AP.PhoneNumber
FROM
    Applicant AS A          -- Alias Applicant table as 'A'
FULL OUTER JOIN
    ApplicantPhone AS AP ON A.ApplicantID = AP.ApplicantID; -- Join based on ApplicantID

GO
```

100 %

Results | Messages

	ApplicantID	FirstName	LastName	PhoneType	PhoneNumber
1	2001	Jude	Hey	Mobile	07700900001
2	2001	Jude	Hey	Mobile	07700900001
3	2001	Jude	Hey	Mobile	07700900001
4	2001	Jude	Hey	Mobile	07700900001
5	2002	Penny	Lane	Mobile	07700900002
6	2002	Penny	Lane	Mobile	07700900002
7	2002	Penny	Lane	Mobile	07700900002
8	2002	Penny	Lane	Mobile	07700900002
9	2003	Rocky	Raccoon	NULL	NULL
10	2004	Eleanor	Vance	Mobile	07700900004
11	2004	Eleanor	Vance	Mobile	07700900004
12	2004	Eleanor	Vance	Mobile	07700900004
13	2004	Eleanor	Vance	Mobile	07700900004
14	2005	Maxwell	Silver	Mobile	07700900005
15	2005	Maxwell	Silver	Mobile	07700900005
16	2005	Maxwell	Silver	Mobile	07700900005
17	2005	Maxwell	Silver	Mobile	07700900005